Technical Analysis and Strategic Mapping of WordPress Token Gating Solutions

I. Introduction

Purpose

Non-fungible tokens (NFTs) and other cryptographic tokens have unlocked novel methods for controlling access to digital content and communities. Token gating, the practice of restricting access based on the ownership of specific digital tokens, offers WordPress website owners powerful mechanisms for creating exclusive content sections, members-only areas, community portals, and unique digital experiences.[1] As Web3 technologies mature, integrating these capabilities into established platforms like WordPress becomes increasingly relevant.

This report provides a detailed technical analysis and comparison of viable token gating solutions for WordPress platforms. The objective is to evaluate the features, security implications, implementation feasibility, and strategic suitability of various approaches, including dedicated WordPress plugins, the Lit Protocol Software Development Kit (SDK) and its associated plugin, potential implementations leveraging Zora's infrastructure, and custom integrations using the WordPress REST API. The analysis places a primary focus on security robustness, recognizing the critical importance of protecting access to potentially valuable gated resources.

Scope

The scope of this investigation encompasses several distinct solution categories:

1. **Free and Freemium WordPress Plugins:** Analysis of readily available plugins from the WordPress ecosystem, including open-source options found on platforms like GitHub.[5]
2. **Lit Protocol:** Evaluation of the Lit Protocol SDK for its decentralized access control capabilities [9] and assessment of its dedicated WordPress plugin.[12]
3. **Zora Infrastructure:** Exploration of how Zora's L2 network and tooling might be utilized for token gating, differentiating between native services and potential custom builds.[13]
4. **Custom WordPress REST API Integration:** Assessment of building bespoke token gating logic using WordPress's native REST API (wp-json) in conjunction with wallet connection libraries like MetaMask or WalletConnect.[16]

These solutions are evaluated against a consistent set of criteria: core features, security robustness (including resistance to common vulnerabilities), ease of implementation, flexibility and customization potential, associated costs (development and operational), maintenance overhead, and the availability of community support and documentation.

Target Audience

This report is intended for technical decision-makers, including Chief Technology Officers (CTOs), Lead Developers, System Architects, and Product Managers, who are responsible for evaluating and implementing Web3 functionalities, specifically token gating, on WordPress-based platforms. A foundational understanding of WordPress architecture and Web3 concepts (blockchain, NFTs, wallets, digital signatures) is assumed.

Methodology

The analysis presented herein is based on a review of provided technical documentation, code repositories (where

available), security best practices outlined by organizations like OWASP [19], and comparative evaluation grounded in the defined criteria. The findings from various sources are synthesized to provide a cohesive assessment of each approach and its strategic implications.

II. WordPress Token Gating Plugins Landscape

Overview

Utilizing dedicated WordPress plugins represents a common and often appealing approach for integrating token gating functionality. The primary allure lies in the potential for simplified setup and seamless integration within the existing WordPress administrative environment and ecosystem.[1] The market offers a spectrum of options, broadly categorized into free, premium, and freemium models, with varying degrees of open-source availability.[6] Numerous plugins listed in the official WordPress plugin directory and other marketplaces claim to offer Web3 login or NFT gating capabilities.[6] However, their actual functionalities, security postures, and maintenance levels differ significantly.

Analysis of Specific Free/Open-Source Plugins

An examination of specific free and open-source plugins reveals diverse approaches and limitations.

- **Fungate [5]:**
  - **Core Functionality:** Fungate positions itself as a tool for enabling Web3 Login and NFT-based content gating directly within WordPress.[5] It utilizes WordPress shortcodes ([fungate]...[/fungate]) to protect content sections based on ownership criteria related to a specific NFT collection contract address, minter address, or individual NFT ID.[5] The plugin is built using PHP and JavaScript and operates directly on the WordPress server.[5] Authentication relies on WalletConnect.[5]
  - **Verification Mechanism:** A critical aspect is its verification process. Fungate uses the Loopring REST API to authenticate users and verify NFT ownership after the initial public key verification step handled by the user's browser or WalletConnect.[5] To mitigate the risk of exposing the Loopring API key directly to the client-side, Fungate implements a local proxy server.[5] This proxy handles requests to the Loopring API, theoretically shielding the key. The verification process involves the user signing a message to prove ownership of their Ethereum address; the plugin explicitly states it cannot initiate token transfers.[5]
  - **Setup & Configuration:** Installation involves downloading the plugin zip file, uploading and activating it within WordPress, and subsequent configuration.[5] This configuration requires obtaining a Loopring API key (exported from the user's Loopring account) and adding it to the Fungate settings page in the WordPress admin dashboard. Additionally, a ProjectId from WalletConnect's cloud service must be obtained and saved in the settings.[5] While documentation and Discord support are offered [5], this multi-step configuration involving external API keys adds complexity compared to simpler plugins.
  - **Limitations & Dependencies:** Fungate's primary limitation is its tight coupling with the Loopring ecosystem, relying specifically on the Loopring REST API for verification.[5] This makes it unsuitable for gating based on assets on other blockchains. Its community traction appears limited, as indicated by low GitHub metrics (8 stars, 1 fork as per the snippet).[5] The plugin is licensed under GPL-3.0.[5] Its functionality is entirely dependent on the availability and stability of the Loopring API and WalletConnect services.
  - **Security Note:** The security hinges on the correct implementation of the local proxy for API key protection and the robustness of the Loopring API itself. The use of message signing for address verification is a positive step.[5] However, the effectiveness of the proxy against potential vulnerabilities needs careful

scrutiny if considering this plugin.

- **Unlock Protocol Plugin [1]:**
  - **Core Functionality:** This plugin integrates the Unlock Protocol's "Paywall Application" into self-hosted WordPress sites.[1] It allows site administrators to lock content – either entire posts/pages or specific blocks within the Gutenberg editor [25] – making it accessible only to users who possess a valid "Key" (an NFT minted via an Unlock Protocol "Lock").[1] It also supports user login via their Unlock Protocol account or connected Ethereum wallet.[25]
  - **Verification Mechanism:** The plugin verifies key ownership by communicating directly with the Unlock Protocol's infrastructure.[25] It utilizes functions like getHasValidKey to check for valid memberships.[25] When a user logs in or attempts to access gated content, the plugin facilitates connecting their wallet and validates ownership through Unlock's systems.[25] It maps WordPress user accounts (creating subscriber roles for new wallet users) to their crypto wallets.[26] This reliance on Unlock's backend suggests a server-side validation model managed by the protocol itself.
  - **Setup & Configuration:** Implementation requires deploying a "Lock" contract using the Unlock Protocol dashboard (potentially on a test network initially).[25] The plugin is then installed via the WordPress directory.[25] Enabling user registration in WordPress settings is recommended for seamless wallet login.[25] Gating content involves adding the "Unlock Protocol" Gutenberg block and configuring it with the relevant Lock contract address(es) and the corresponding blockchain network.[25] The plugin supports multiple chains (Ethereum, Polygon, etc.) and offers Stripe integration for users without crypto wallets.[26] Customization options for login/checkout buttons are available.[25]
  - **Open Source & Community:** The plugin is presented as open-source, with direct links to its GitHub repository.[1] It is developed and maintained by Unlock Inc., the entity behind the protocol [1], suggesting a reasonable level of support and alignment with the core protocol's development. The repository likely uses an Apache-2.0 license, common for Unlock projects.
  - **Limitations & Dependencies:** The primary dependency is the Unlock Protocol itself. Functionality is tied to Locks created and managed through their platform, and verification relies on their infrastructure.[25] Users interact with the standard Unlock checkout user interface.[25]
- **Web3 Token Gate (by Abraxas Wizard / W3P) [7]:**
  - **Core Functionality (Free Version):** The free tier of this plugin offers limited gating capabilities. It allows restricting access to a specific *message* displayed within a WordPress page or post, based on ownership of a particular ERC-721 standard NFT on the Ethereum Mainnet.[23] Verification is handled exclusively via the MetaMask wallet.[23] Administrators can define custom messages for successful verification (revealing the hidden content) and failed verification (e.g., linking to NFT marketplaces).[23] The plugin emphasizes privacy by stating it does not store wallet addresses or user information.[23]
  - **Verification Mechanism:** The plugin authenticates the user and checks their wallet for the specified NFT via MetaMask.[23] Given the constraints (MetaMask only, Ethereum only, message gating only), the verification process in the free version is strongly implied to be client-side, occurring within the user's browser environment.
  - **Setup & Configuration:** Installation follows the standard WordPress plugin procedure.[23] Configuration primarily involves specifying the target Ethereum ERC-721 contract address within the plugin's settings.[23]
  - **Limitations (Free Version):** The free version is significantly restricted: only Ethereum Mainnet, only ERC-721 tokens, only MetaMask wallet support, and critically, it can only gate a single message block, not

entire posts, pages, or other content types.[23]

  - **Pro Version Differences:** The paid "Pro" version substantially expands functionality, addressing the free version's limitations.[23] It enables gating of specific content sections, full pages, posts, and videos. It adds support for multiple shortcodes, allowing different content areas to be gated by different NFT contracts. Wallet support is broadened to include WalletConnect, Coinbase, TrustWallet, and others. Blockchain compatibility extends to Polygon (Matic), Binance Smart Chain (BSC), and potentially more. Caching options are introduced, particularly for WooCommerce integration. Crucially, it adds support for the ERC-1155 token standard alongside ERC-721.[23]

- **Lit Protocol WordPress Plugin ("Token / NFT / Blockchain Page Gating") [6]:**
  - **Core Functionality:** This plugin aims to gate entire WordPress pages based on blockchain conditions, explicitly mentioning NFT ownership and DAO membership as examples.[12] It leverages the underlying Lit Protocol for enforcing these conditions [12] and supports Ethereum and most other EVM-compatible chains.[12]
  - **Verification Mechanism:** Verification relies on the Lit Protocol's decentralized network.[12] The process likely involves the user connecting their wallet, the plugin defining Access Control Conditions (ACCs) based on the page's settings, and the client interacting with the Lit network. The Lit network nodes would verify if the user's wallet meets the ACCs and grant access (potentially by providing a decryption key or signing a message) if conditions are met.[10]
  - **Setup & Configuration:** Installation is standard. A notable requirement is disabling the "AMP" WordPress plugin, as it conflicts with the gating mechanism on mobile devices.[12] Configuration would involve defining the specific blockchain conditions (e.g., required NFT contract address, token ID, DAO membership criteria) that grant access to a particular page, likely translating these into Lit Protocol ACCs.
  - **Status & Maintenance:** The plugin is listed as free.[6] However, its maintenance status raises concerns. The last update recorded was in September 2022, and it was tested only up to WordPress version 5.9.10.[12] The active installation count is very low (30).[12] While a GitHub source code link is provided [12], access to this repository might be problematic.[27] These factors strongly suggest the plugin may be outdated, unmaintained, and potentially incompatible with current WordPress versions or Lit Protocol updates. This significantly diminishes its viability as a reliable solution.
  - **Limitations:** Beyond the likely lack of maintenance, the plugin depends on the Lit Protocol network's availability and performance. The incompatibility with the AMP plugin is a practical limitation for mobile-focused sites.[12]

Analysis of Premium/Freemium Plugins

Premium solutions often provide more comprehensive features and dedicated support, albeit at a cost.

- **miniOrange Web3 Authentication [2]:**
  - **Core Functionality:** This plugin offers a suite of Web3 features, starting with crypto wallet login in its free version.[17] Supported wallets include MetaMask, WalletConnect, Coinbase, Phantom, MyAlgo, and even cold storage wallets like Ledger and Trezor in premium tiers.[17] The core value proposition for token gating resides in its premium versions.[17] These tiers enable gating of entire pages, posts, videos, or specific sections within content based on NFT ownership.[17] It supports multiple blockchains (Ethereum, Polygon, BSC, Hedera, Algorand, Solana) and various token standards (ERC-20, ERC-721, ERC-1155, BEP-20/721/1155, SPL, Algorand Asset IDs, etc.).[17] Premium features also include mapping WordPress user

roles based on owned NFTs and integrations with popular plugins like WooCommerce, MemberPress, and BuddyBoss.[17]

  - **Verification Mechanism:** User login is secured through digital signature verification.[17] The plugin documentation explicitly mentions server-side verification of these signatures.[17] To prevent replay attacks during login, it employs database nonces that are refreshed upon use and appends the user's IP address or Wallet Address to the signed message.[17] For content gating, the verification involves checking the connected wallet's holdings against the configured NFT requirements (contract address, specific token IDs, minimum token count) defined by the administrator.[2]
  - **Setup & Configuration:** Installation follows the standard WordPress plugin process.[17] Configuring token gating requires accessing the miniOrange Web3 authentication plugin dashboard within WordPress.[2] Administrators define the gating rules by adding token details (blockchain, contract address, optional token counts or specific IDs) and then applying these rules to specific content using Page URL regular expressions and defining an error URL for users who fail verification.[2]
  - **Cost:** While basic wallet login is free, all NFT token gating functionalities necessitate purchasing a premium license.[17] Detailed pricing plans are available on the miniOrange website.[24]
  - **Security Note:** The explicit mention of server-side signature verification, use of nonces, and IP/address checks for login [17] indicates a more security-conscious design compared to purely client-side approaches. The overall security depends on the robustness and correctness of these server-side implementations. As a proprietary plugin, direct code auditing is not possible.
- **Other Mentioned Plugins:** Several other plugins touch upon related Web3 functionalities but may not be direct token gating solutions. Auth0 [32] provides general authentication (SSO, MFA) but isn't specifically for token gating. EthPress [8] focuses on Web3 login. Web3Press [6] allows publishing posts as NFTs. Various NFT Gallery plugins [6] are for displaying NFTs rather than gating content based on ownership. It's crucial to differentiate between plugins offering true content restriction and those focused solely on login, display, or minting.

Key Considerations and Trends in the Plugin Landscape

Analyzing the available plugins reveals several important patterns and considerations:

- **Prevalence of Freemium Models:** A common pattern emerges where free plugin versions offer basic Web3 login or highly restricted gating (e.g., specific message types, limited chains/wallets).[17] Truly functional and flexible token gating, encompassing various content types, multiple blockchains and token standards (like ERC-1155 or SPL), diverse wallet support, and integrations with other WordPress components (like WooCommerce or membership plugins), consistently requires upgrading to a paid or premium tier.[17] This suggests that while free options can serve as an introduction, practical and robust implementations typically involve a financial investment in a premium plugin license.
- **Diversity and Security Implications of Verification Methods:** The mechanisms plugins use to verify token ownership vary significantly, impacting their security and reliability. Some rely on external, third-party APIs (like Fungate using the Loopring API via a proxy [5]), introducing dependencies and potential vulnerabilities related to the API or the proxy itself. Others leverage a specific protocol's infrastructure (like the Unlock Protocol plugin using Unlock's backend for validation [25]). Some free versions appear to rely solely on client-side checks performed in the user's browser [23], which are inherently insecure and easily bypassed (detailed further in Section VI). In contrast, solutions like miniOrange explicitly state the use of server-side signature verification [17], a generally more secure approach. This diversity underscores that choosing a "plugin"

is not a uniform decision regarding security; the specific implementation details of the verification process are paramount.

- **Trade-offs Between Open Source and Maintenance:** While genuinely open-source options exist, such as Fungate [5] and the Unlock Protocol plugin [1], their development activity, community support, and long-term maintenance can vary. Fungate shows limited GitHub activity [5], while the Lit Protocol plugin appears potentially abandoned.[12] Unlock, backed by its parent organization [1], likely benefits from more consistent upkeep. Conversely, proprietary premium plugins like miniOrange [28], developed by commercial entities, may offer dedicated support channels and regular updates but lack the transparency of open-source code, preventing independent security audits by the user. This presents a fundamental trade-off: open-source offers transparency and potential for customization but may lack guaranteed support, while proprietary solutions offer support but limit visibility into the underlying code.

Feature and Security Comparison of Key WordPress Token Gating Plugins

To provide a clearer overview, the following table summarizes key aspects of the discussed plugins:

Feature	Fungate	Unlock Protocol	Web3 Token Gate (Free)	Web3 Token Gate (Pro)	Lit Protocol WP Plugin	miniOrange (Free)	miniOrange (Premium)
Developer	stepwn	Unlock Inc.	Abraxas Wizard / W3P	Abraxas Wizard / W3P	LitProtocol.com	miniOrange	miniOrange
License	GPL-3.0	Apache-2.0 (likely)	Proprietary (likely)	Proprietary (likely)	Unknown (GitHub repo [12])	Proprietary	Proprietary
Free Features	Login (WC), NFT Gating (Loopring)	Login (Unlock), NFT Gating (Unlock Keys)	Login (MetaMask), Message Gating (ETH ERC721)	N/A	NFT/DAO Page Gating (EVM)	Login (MetaMask, WC, etc.)	N/A
Premium Features	N/A	N/A (Core protocol is free)	N/A	Full Content Gating, Multi-Chain/ Wallet/Standard, Cache, etc.	N/A	N/A	Full Content Gating, Multi-Chain/ Wallet/Standard, Role Mapping, Integrations
Supported Chains	Loopring (Ethereum L2)	ETH, Polygon, Gnosis, etc.	Ethereum Mainnet	ETH, Polygon, BSC, etc.	ETH, most EVM chains	ETH, Polygon, BSC, etc. (Login)	ETH, Polygon, BSC, Hedera, Algo, Solana, etc.
Supported	Loopring	Unlock Keys	ERC-721	ERC-721,	NFT, DAO (via Lit	N/A (Login	ERC-20, 721, 1155, BEP,

Tokens	NFTs	(NFTs)		ERC-1155	ACCs)	only)	SPL, etc.
Supported Wallets	WalletConnect	MetaMask, WalletConnect, etc.	MetaMask	MetaMask, WC, Coinbase, etc.	WalletConnect, MetaMask (Implied by Lit)	MetaMask, WC, Coinbase, Phantom, MyAlgo	All Free + Cold Wallets (Ledger, Trezor)
Verification Method	Server-Side (via Loopring API & Proxy)	Server-Side (via Unlock Infra)	Client-Side (Likely)	Server-Side (Likely for Pro features)	Client-Side (via Lit Network)	Server-Side (Signature for Login)	Server-Side (Signature + Ownership Check)
Key Security Measures	Local Proxy for API Key, Msg Signing	Unlock Protocol Security	None Noted (Privacy claim)	Unknown	Lit Protocol Security	Signature Verification, Nonce	Signature Verification, Nonce, Ownership Checks
Maintenance Status	Low Activity (8 stars)	Actively Maintained	Active (Updated Apr 2024)	Active (Updated Apr 2024)	Likely Unmaintained (Updated Sep 2022)	Active	Active
Ease of Setup	Medium (API Keys needed)	Medium (Lock creation needed)	High	Medium	Medium (AMP conflict, Lit setup?)	High	Medium (Gating config needed)

Note: Ratings for Ease of Setup are subjective. Verification methods for some plugins are inferred based on features and documentation.

III. Lit Protocol SDK for Access Control

Introduction to Lit Protocol

Lit Protocol presents a fundamentally different paradigm for access control, operating as a decentralized key management and compute network.[9] Instead of relying on a centralized server or a specific plugin's logic, Lit utilizes threshold cryptography distributed across a network of nodes.[9] For token gating, its core value lies in two primary mechanisms: defining Access Control Conditions (ACCs) to manage encryption and decryption of data [10], and enabling programmatic signing based on verified conditions, often through JSON Web Tokens (JWTs) or Lit Actions.[33] A key advantage is its blockchain-agnostic design, supporting interactions with most EVM-compatible chains, Solana, and Cosmos networks.[9]

Access Control Mechanisms

Lit Protocol offers sophisticated methods for implementing token gating logic:

- **Encryption/Decryption based on ACCs:** This is a primary mechanism where content (text, video URLs, files, etc.) is encrypted on the client-side using a symmetric key. This symmetric key is then encrypted by the Lit Protocol network, and access to decrypt it is tied to specific Access Control Conditions (ACCs).[10] These ACCs define the rules that a user must satisfy to gain decryption capabilities. Conditions can range from owning a

specific NFT (ERC-721 or ERC-1155) [10], being a member of a particular DAO [10], matching the result of a smart contract function call [10], or even satisfying conditions based on off-chain data verified via Lit Actions.[10] When a user attempts to access the encrypted content, they present the ACCs to the Lit network nodes. The nodes independently verify if the user's connected wallet meets these conditions. If a sufficient threshold of nodes agrees the conditions are met, they collectively release shares of the decryption key, allowing the user's client to reconstruct the key and decrypt the content.[11] The encrypted content itself can be stored anywhere – IPFS, Arweave, or even centralized storage like AWS S3.[11]

- **JWT Authentication based on ACCs [36]:** Lit Protocol can also facilitate server-side gating through signed JWTs. In this flow, a web application defines ACCs required to access a specific resource (e.g., a protected API endpoint or webpage). When a user attempts access, their client requests a JWT from the Lit network, providing the necessary ACCs.[36] The Lit nodes verify if the user meets these conditions. If successful, the network generates and signs a JWT using its distributed BLS key. This JWT contains the verified ACCs as claims within its payload.[36] The client then presents this Lit-issued JWT to the application server. The server's responsibility is to verify the JWT's signature using the Lit network's public key and to check that the ACCs listed in the JWT's claims match the requirements for the requested resource.[36] If both checks pass, the server grants access. This mechanism allows secure server-side authorization without the server needing direct wallet interaction or complex blockchain state querying, as it relies on Lit's decentralized attestation.
- **Lit Actions for Dynamic Conditions [33]:** Lit Actions are immutable JavaScript functions executed by the Lit nodes within a secure environment.[33] These actions can fetch and process data from arbitrary on-chain or off-chain sources (e.g., APIs, databases).[33] Lit Actions can be incorporated into ACCs, enabling highly dynamic and complex gating logic. For example, access could be granted based on a combination of NFT ownership *and* the result of an external API call checked by a Lit Action.[10]

Integration Requirements & SDK Usage

Integrating Lit Protocol requires development effort using its JavaScript SDK.

- **SDK Installation & Setup:** Developers need to install the appropriate Lit JS SDK package, typically @lit-protocol/lit-node-client for browser environments or @lit-protocol/lit-node-client-nodejs for server-side use.[9] The client application must initialize and connect to the Lit network nodes using litNodeClient.connect().[9]
- **Defining Access Control Conditions (ACCs):** ACCs are defined as JSON objects specifying the chain, contract address, function name (e.g., balanceOf), parameters, and expected return value comparison.[11] Lit supports combining multiple conditions using boolean operators (AND, OR) for complex logic.[11]
- **Encryption/Decryption Flow:** The SDK provides functions like encryptString and decryptString.[38] To authorize decryption requests or save encryption keys (saveEncryptionKey, getEncryptionKey), the user must authenticate with the Lit network. This is typically done by signing an authentication message (checkAndSignAuthMessage) to obtain an authSig or, preferably, using sessionSigs for more persistent sessions.[38]
- **JWT Authentication Flow:** The client first obtains sessionSigs from the user to authenticate with the Lit network.[36] Then, the getSignedToken function is called, passing the required ACCs and the sessionSigs, to request the signed JWT from the Lit network.[36] On the server-side, the verifyJwt function is used, requiring the received JWT and the Lit network's public key, to validate the token's authenticity and extract its payload for claim verification.[36]

Costs, Fees, and Limitations [11]

Using Lit Protocol involves potential costs and has certain limitations.

- **Payment Mechanism:** Certain Lit network operations, specifically those requiring computation or signing by the nodes (like decryption, JWT signing via getSignedToken, and Lit Action execution), require payment.[42] Connecting to the network, generating session signatures, and encrypting data are generally free.[42] Payment is managed through **Capacity Credits**.[42]
- **Capacity Credits:** These are NFT-based tokens representing a reservation of computational capacity on the Lit network, defined as a certain number of requests per second over a specified duration.[42] Developers mint these credits on the Lit Chronicle Yellowstone (testnet currently) blockchain using the native test token testLPX (obtainable from a faucet).[42] When making a paid request to the Lit network, the ID of a valid Capacity Credit must be provided.[43] Payment can also be managed via a Payment Delegation Database, where a Payer Wallet funds usage for designated Payee users.[42]
- **Costs:** The *specific cost* of minting Capacity Credits or the per-operation cost (e.g., cost per decryption or JWT signature request) is *not explicitly detailed* in the provided documentation snippets.[11] While the existence of a payment mechanism is clear [42], the exact pricing structure requires consulting further documentation on Capacity Credits [43] or the Lit Explorer.[43] The pre-sale information for the $LITKEY token suggests a future mainnet model where this token will be used for payment, but details on SDK usage fees separate from the token are absent.[41] It's important to note that other LIT tokens mentioned [39] appear unrelated to the protocol's operational token.
- **Rate Limits:** Explicit rate limits are not defined, but the Capacity Credit system itself implies a rate limit based on the reserved capacity of the credit being used.[42] The Lit Relayer, an auxiliary service for onboarding, might be subject to rate limiting due to its shared nature.[42]
- **Limitations:** Implementing Lit Protocol is inherently more complex than using a pre-built WordPress plugin, requiring JavaScript development skills and a solid understanding of Web3 concepts. The system's functionality depends on the Lit network's availability, performance, and associated operational costs via Capacity Credits.[42]

Lit Protocol WordPress Plugin [6]

While Lit Protocol offers a dedicated WordPress plugin ("Token / NFT / Blockchain Page Gating") [12], its practical viability is questionable.

- **Features:** It aims to gate WordPress pages based on blockchain conditions (NFT ownership, DAO membership) using Lit Protocol's ACCs, supporting EVM chains.[12]
- **Status:** Although listed as free [6], the plugin appears unmaintained. Its last update was in September 2022, it's tested only up to WordPress 5.9.10, has a very low active installation count (30), and conflicts with the AMP plugin on mobile.[12] Access to its GitHub repository may also be restricted.[27] This combination of factors strongly suggests it's not a reliable option for current WordPress installations, likely requiring a custom SDK integration instead.

Key Implications of Using Lit Protocol

The Lit Protocol approach offers distinct advantages and disadvantages compared to traditional plugin methods:

- **Decentralized and Flexible Verification:** Lit provides a fundamentally different model by using its decentralized network of nodes for verifying access conditions.[9] The use of ACCs allows for highly flexible and composable rules, combining various on-chain states (NFT/token ownership, contract results, DAO membership) and even off-chain data (via Lit Actions) with boolean logic.[10] This offers significantly more power and avoids reliance on a single plugin's specific API integrations or potentially centralized verification infrastructure.[1]
- **Enhanced Server-Side Security via JWTs:** The JWT authentication mechanism [36] represents a robust method for implementing secure server-side gating. The application server doesn't need direct blockchain access or handle private keys; it simply verifies the cryptographically signed attestation (the JWT) provided by the decentralized Lit network.[36] This cryptographic verification, combined with checking the claims within the JWT against server-side requirements, offers a strong security posture compared to client-side checks or server-side methods requiring direct RPC node management.
- **Significant Cost and Complexity Trade-off:** The power and security benefits of Lit come at the cost of increased complexity and potential operational expenses. Direct SDK integration requires proficient JavaScript development.[9] Furthermore, utilizing core network functions like decryption and signing necessitates payment through Capacity Credits [42], introducing ongoing operational costs that differ from the one-time or subscription fees of plugins.[17] The apparent lack of a reliably maintained WordPress plugin [12] means that realizing Lit's benefits on WordPress likely requires a custom integration effort, further increasing upfront development time and cost.

IV. Zora's Token Gating Implementation

Introduction to Zora

Zora operates primarily as an NFT marketplace protocol and has launched its own Layer 2 (L2) network built on the OP Stack, designed to be a fast and cost-efficient environment for creators and NFT-related activities.[14] Developers can deploy smart contracts to the Zora Network using standard EVM tools like Foundry and Hardhat.[13] It's noteworthy that the announced $ZORA token is explicitly positioned as a "fun token" for community engagement, not granting governance rights or equity.[45]

Documented Token Gating Features

The available documentation suggests Zora supports token gating use cases primarily through integrations and standard blockchain functionalities, rather than offering a dedicated native service.

- **Third-Party Integrations (Formo Example) [14]:** Zora's integration with Formo, a Web3 form builder, explicitly enables the creation of NFT-gated forms and surveys.[14] Users can configure Formo forms to require specific token or NFT balances on the Zora network for access.[14] This demonstrates that the Zora network *can* be used as the basis for token gating by external tools, but Formo provides the gating logic itself.
- **Permissions1155 Contract [47]:** Zora's documentation details a Permissions1155 smart contract pattern.[47] This contract includes roles like Admin and Minter and provides functions (addPermission, removePermission, isAdminOrRole) to manage which addresses are allowed to mint specific tokens or manage contract-level settings.[47] While this involves access control, it pertains specifically to *minting permissions* within a Zora-based NFT contract, not general-purpose *content access* gating for a website.
- **Absence of Native Content Gating API [48]:** A review of the provided Zora documentation ([13]) and related search results [15] reveals no specific, publicly documented Zora API endpoint or SDK function designed for

server-side verification of NFT ownership for the purpose of generic website content gating.[48] The developer documentation focuses predominantly on deploying contracts [13], interacting with Zora's minting mechanics (like Premint [50]), managing metadata [49], and understanding protocol rewards.[49]

Server-Side vs. Client-Side Verification Context

Understanding the distinction between server-side and client-side verification is crucial when considering how token gating might be implemented using Zora assets.

- **Client-Side Verification:** This approach performs checks within the user's browser environment.[53] Typically, JavaScript code connects to the user's wallet, retrieves the address, and queries token balances either through the wallet's provider or direct RPC calls.[53] The primary risk is its vulnerability: since the code runs on the user's machine, it can be inspected, manipulated, or bypassed entirely.[55] This makes client-side gating suitable only for low-security scenarios where bypass risk is acceptable.
- **Server-Side Verification:** This method shifts the verification logic to the backend server.[54] The process typically involves two main steps: first, securely authenticating the user to confirm they control the claimed wallet address (usually via a signed message containing a nonce [17]); second, the server queries the blockchain (using RPC nodes [60] or trusted indexing APIs [60]) to verify if the authenticated address holds the required token(s).[54] This approach is significantly more secure because the verification logic and blockchain interaction are protected on the server, inaccessible to the end-user.[56]
- **Inferred Approach for Zora:** Given that Zora provides L2 infrastructure [14] and supports standard EVM development tools [13], a secure token gating solution built *using* Zora assets would necessitate server-side verification. Developers would need to implement this themselves. They could deploy gating logic contracts on Zora [13] or, more commonly for website gating, use a backend server (like a WordPress site) to perform the checks. This backend would authenticate the user via signed message and then query the Zora network's RPC endpoints (rpc.zora.energy or testnet.rpc.zora.energy [13]) using standard EVM methods like balanceOf or ownerOf to verify NFT ownership. Zora itself, based on current documentation, does not appear to provide a managed service layer to handle this verification.[48]

Advantages and Disadvantages (Hypothetical Zora-based Solution)

Building a token gating solution leveraging the Zora network would have potential benefits and drawbacks:

- **Advantages:**
  - **Low-Cost Transactions:** If the gating logic requires on-chain interactions (less common for simple content access but possible for complex systems), performing these on the Zora L2 could offer lower gas fees compared to Ethereum mainnet.[14]
  - **Ecosystem Integration:** Tightly integrates with Zora's creator-focused tools and community if the gated content is related to assets minted or traded on Zora.
- **Disadvantages:**
  - **Lack of Native Service:** The absence of a specific, documented Zora API or managed service for content gating means the developer bears the full responsibility for building and maintaining the verification logic.[48]
  - **Implementation Effort:** Requires significant development effort to implement secure server-side authentication (signature verification) and blockchain state querying (RPC/API calls).

- **Network Specificity:** Gating would naturally be easiest for assets residing on the Zora network. Implementing gating based on assets on other chains would require additional cross-chain verification mechanisms.

Key Implications Regarding Zora

The analysis suggests two main points regarding Zora and token gating:

- **Zora as Infrastructure, Not a Dedicated Gating Service:** Zora primarily provides the underlying L2 blockchain infrastructure [14] and compatibility with standard EVM development tools.[13] While this infrastructure *enables* developers to build token gating solutions for Zora-based assets, Zora itself does not currently appear to offer a specific, managed API or service dedicated to this purpose, unlike Lit Protocol's ACCs/JWTs or the focused functionality of certain WordPress plugins. The Formo integration [14] exemplifies how third-party services build *on top of* Zora's infrastructure rather than using a native Zora gating API. The focus of Zora's own documentation remains on minting, contract deployment, and network details [13], with no endpoints like zora.verifyNFTOwnership being described.[48]
- **Developer Responsibility for Verification:** Consequently, implementing token gating using Zora assets places the burden of secure verification squarely on the developer. This involves not only writing the frontend code for wallet connection and message signing but also building the backend logic for securely verifying the signature (with nonce protection) and querying the Zora network's state via RPC calls [13] or utilizing third-party indexing services (like Alchemy [61], QuickNode [63], Moralis [64], SimpleHash [51], Bitquery [60]) that support Zora's EVM-compatible network. This contrasts sharply with solutions like Lit Protocol [36] or the Unlock Protocol plugin [25], which abstract away significant parts of the verification process.

V. Custom Integration via WordPress REST API

Concept

An alternative to relying on pre-built plugins or external services is to construct a bespoke token gating solution by extending WordPress's native capabilities. This involves leveraging the built-in WordPress REST API (accessible via /wp-json/ endpoints [16]) to create custom server-side logic that interacts with Web3 wallets like MetaMask or WalletConnect [17] to verify token ownership and control access to content.

Implementation Pattern

A typical implementation pattern for custom REST API-based token gating involves distinct frontend and backend components:

- **Frontend (Client-Side):**
  1. **Wallet Connection:** The user connects their preferred Web3 wallet (e.g., MetaMask browser extension, WalletConnect modal) to the WordPress site.[17] Standard JavaScript libraries (like ethers.js, web3.js, or dedicated WalletConnect SDKs) handle this interaction.
  2. **Nonce Request:** The frontend requests a unique, single-use nonce from a dedicated WordPress backend endpoint. This nonce is crucial for preventing replay attacks.[17]
  3. **Message Signing:** The user is prompted to sign a specific message using their connected wallet.[59] This message *must* include the server-provided nonce and should clearly state the purpose (e.g., "Sign in to ExampleSite with nonce: 123abc789"). Signing proves the user controls the private key associated with the

connected public address.

4. **API Request:** The frontend sends a request (typically POST) to a custom WordPress REST API endpoint. This request includes the user's public wallet address, the original message (with the nonce), and the resulting signature.[69]

- **Backend (Custom WordPress REST Endpoint):**
  1. **Endpoint Registration:** A custom REST API route and endpoint are registered within WordPress using the register_rest_route function, typically hooked into the rest_api_init action.[69]

     PHP
     ```php
     add_action( 'rest_api_init', function() {
       register_rest_route( 'my-token-gate/v1', '/verify-ownership', array(
         'methods' => 'POST', // Use POST to receive data
         'callback' => 'handle_token_verification_request',
         'permission_callback' => '__return_true' // Public endpoint initially, auth happens via signature
         // Add argument definitions for address, message, signature, nonce
       ) );
     } );
     ```

  2. **Nonce Validation:** Inside the handle_token_verification_request callback function, the first step is to validate the received nonce. The server must check if the nonce is valid, hasn't been used before, and potentially matches the user's current session. Invalid or reused nonces must result in rejection.[17]
  3. **Signature Verification:** If the nonce is valid, the core cryptographic verification occurs. The PHP backend needs to verify that the provided signature corresponds to the original message and the claimed public address.[59] This requires using a PHP library capable of performing Ethereum signature verification (specifically, the ecrecover operation). Several libraries exist, such as simplito/elliptic-php [72], agustind/ethsignature [76], wmh/php-ecrecover [77], or potentially lower-level implementations using phpseclib [78] or custom code.[79] It is critical to ensure the message hashing conforms to the Ethereum standard ("\x19Ethereum Signed Message:\n" + message length + message) before verification.[78] If the signature is invalid, the request is rejected.
  4. **Token Ownership Check:** Upon successful signature verification (confirming the user controls the address), the server must check if this address owns the required NFT or token.[59] This check *must* be performed server-side. It involves querying a blockchain node via its JSON-RPC URL (using PHP libraries like digitaldonkey/ethereum-php [81] for abstraction or direct HTTP requests) or, more efficiently, using a trusted third-party indexing API service (e.g., Alchemy [61], QuickNode [63], Moralis [64]). The specific contract function to call depends on the token standard: balanceOf(address) for ERC-721/ERC-1155 [82] or ownerOf(tokenId) for ERC-721.[82] If the address does not hold the required asset, the request is rejected.
  5. **Access Grant / Session Management:** If both signature and ownership are verified, the server grants access. The method for granting access depends on the application architecture:
     - **Simple Status Return:** The API endpoint returns a success status. The frontend then reveals the pre-loaded gated content. This is less secure if sensitive content is loaded client-side before verification.
     - **Server-Side Session/Cookie:** The server establishes a standard WordPress session or sets a secure, HttpOnly cookie [86] associated with the verified address or corresponding WordPress user ID.[59] Subsequent requests for gated pages or resources check for this valid session/cookie server-side before rendering content. This is a common and generally secure pattern for traditional web applications.

- **JWT Issuance:** The server generates a JSON Web Token (JWT) [87] signed with a server-side secret key. This JWT contains claims identifying the verified user (address/ID), permissions, and an expiration time. The JWT is sent back to the client. For subsequent requests to gated REST API endpoints or content-serving routes, the client includes the JWT in the Authorization: Bearer <token> header.[66] The server validates the JWT signature and claims before granting access. WordPress plugins exist to facilitate JWT generation and validation.[6]

Authentication Considerations

While the core of this approach relies on cryptographic signature verification, integration with WordPress's standard authentication mechanisms needs consideration. Standard REST API authentication methods like Application Passwords [16] or OAuth [92] might be used for controlling access to the custom endpoint itself, although making the verification endpoint initially public and relying solely on signature/nonce validation is common. WordPress's built-in nonce system [86] should be leveraged server-side for generating and validating the nonce used in the signed message to prevent CSRF and replay attacks effectively.

Required Expertise & Effort

This custom approach demands a high level of technical expertise. Developers must be proficient in:

- **PHP Development:** Deep understanding of WordPress hooks (rest_api_init), the REST API framework (register_rest_route, WP_REST_Request, WP_REST_Response), potentially Composer for managing PHP dependencies.[72]
- **JavaScript/Frontend Development:** Implementing wallet connection flows (MetaMask, WalletConnect [18]), message signing interactions, and handling API communication with the WordPress backend.
- **Web3 Cryptography:** Understanding Ethereum message signing standards and the ecrecover process for signature verification.[68]
- **Security Best Practices:** Implementing robust nonce generation and validation, secure handling of blockchain interactions (RPC/API calls), and secure session or JWT management.[59]

The development effort is significantly higher compared to installing and configuring a plugin.

Examples & Libraries

While no single snippet provides a complete end-to-end example for WordPress REST API token gating with wallet signature verification, relevant components and libraries include:

- **Custom WP REST Endpoint Creation:** Gists and documentation show the basic structure.[69]
- **PHP Ethereum Signature Verification:** Several libraries are available, including agustind/ethsignature [76], simplito/elliptic-php [72] (noteworthy dependency on GMP/bcmath extensions [96], potential maintenance questions [97]), wmh/php-ecrecover [77], digitaldonkey/ecverify [80], and the SIWE-focused zbkm/siwe.[95] StackOverflow discussions often point to implementation challenges and specific code examples.[78] Careful vetting of library maintenance status and security is essential.
- **WalletConnect Integration:** General JavaScript examples exist [18], but direct integration with a custom WP REST API flow requires custom frontend code.
- **WP REST API Authentication (JWT/OAuth):** Plugins like miniOrange [30] or dedicated JWT plugins [6] could potentially be used to manage the session/token *after* successful signature and ownership verification by the

custom callback, but they don't perform the core Web3 verification themselves.

Key Implications of the Custom REST API Approach

Choosing to build a custom solution carries significant implications:

- **Maximum Flexibility, Maximum Effort:** This path offers unparalleled control over every aspect of the token gating process – the user interface for wallet connection and signing, the specific verification logic (multiple tokens, complex conditions), the method of granting access (session, JWT, etc.), and integration with other custom WordPress features. However, this flexibility comes at the price of the highest development effort, requiring the team to build and test all components from the ground up.[69]
- **Security Burden Shifts Entirely to the Developer:** Unlike using a vetted plugin or a service like Lit Protocol, the security of the entire token gating mechanism rests solely on the quality and correctness of the custom implementation.[59] Any flaw in signature verification logic [79], nonce handling [17], interaction with blockchain nodes/APIs, or session management can introduce critical vulnerabilities.[21] The developer must possess strong security expertise and diligently follow best practices.
- **PHP Web3 Ecosystem Considerations:** The PHP ecosystem for Ethereum interactions, particularly for robust and actively maintained signature verification libraries, appears somewhat less mature or standardized compared to the JavaScript (Node.js) ecosystem dominated by ethers.js and web3.js.[72] Developers might encounter challenges finding well-documented, secure, and dependency-light libraries, necessitating careful evaluation, testing, and potentially contributing to or maintaining chosen libraries. Issues with dependencies like GMP or bcmath [96] or outdated libraries [98] can add friction.

VI. Security Analysis of Token Gating Solutions

Importance of Security

Token gating mechanisms act as digital gatekeepers, controlling access to potentially exclusive or valuable content, features, or communities. Consequently, the security and robustness of the chosen implementation are paramount. A compromised gating system can lead to unauthorized access, dilution of exclusivity, revenue loss (if gated content is monetized), and significant damage to user trust and brand reputation. Therefore, a thorough understanding and mitigation of potential security vulnerabilities are critical.

Common Vulnerabilities & Attack Vectors

Several attack vectors specifically target token gating implementations or the underlying Web3/API technologies they rely upon:

- **Client-Side Manipulation/Bypass [57]:**
  - **Description:** This is arguably the most fundamental vulnerability in poorly designed gating systems. If the logic that verifies token ownership (e.g., checking wallet connection status, calling balanceOf via JavaScript) executes entirely within the user's web browser, it is susceptible to manipulation.[56] An attacker doesn't need to steal the required NFT; they can simply modify the client-side JavaScript code executing in their browser, interfere with API calls made by the script, or alter the browser's state to make the script believe the conditions are met. Tools like browser developer consoles make this relatively straightforward for technically savvy users.
  - **Relevance:** This directly affects any token gating implementation that relies solely on frontend checks without server-side validation. This includes naive custom implementations and potentially some free or poorly designed plugins. The accessibility of client-side code is a core tenet of client-side security risks.[56]

- **Mitigation:** The definitive mitigation is to perform all critical verification logic on the server-side.[54] The client should only be responsible for initiating the process (connecting wallet, signing message) and receiving the access decision from the server.
- **Wallet Address Spoofing / Address Poisoning [101]:**
  - **Description:** This attack involves scammers sending negligible amounts of cryptocurrency or worthless tokens from a wallet address crafted to look visually similar (e.g., same starting and ending characters) to an address the victim frequently interacts with.[101] The goal is to "poison" the victim's transaction history, hoping they will later mistakenly copy the scammer's address from the history when intending to send funds to the legitimate address.[101]
  - **Relevance:** While primarily a threat targeting users sending funds, it highlights risks related to address handling. For token gating, if a system were to insecurely rely *only* on a user *inputting* their address without cryptographic proof (signing), spoofing could be a theoretical vector. More practically, it underscores the importance of relying on cryptographic authentication (signatures) rather than user-provided identifiers and the need for user education if the gating process involves directing users to purchase required NFTs.
  - **Mitigation:** For gating systems, always verify wallet ownership via a cryptographic signature challenge.[59] For users, best practices include never copying addresses from transaction history, using ENS or similar naming services [101], double-checking the *entire* address before any transaction [101], ignoring unsolicited "dust" transactions or tokens [102], and utilizing wallet features like address books or transaction warnings.[103]
- **Signature Replay Attacks [55]:**
  - **Description:** If a user signs a message to authenticate or verify ownership for gating purposes, an attacker might intercept this signed message. If the system doesn't prevent reuse, the attacker could "replay" this valid signature at a later time to gain unauthorized access themselves.[55]
  - **Relevance:** This is a critical threat for any system using signed messages for authentication or authorization, including custom REST API solutions and plugins employing signature verification.[17]
  - **Mitigation:** The standard defense is the use of nonces (numbers used once). The server should generate a unique, unpredictable nonce for each authentication attempt and include it in the message the user signs.[68] The server must then validate the nonce upon receiving the signed message and ensure it hasn't been used previously.[17] Timestamps within the signed message can also help limit the window for replay.[106] The miniOrange plugin documentation mentions the use of database nonces for this purpose.[17]
- **JWT Vulnerabilities [21]:**
  - **Description:** If JWTs are used for session management after the initial Web3 verification, they introduce their own set of potential vulnerabilities.[21] Common issues include: attackers modifying the header to claim the algorithm is none (bypassing signature checks if the library allows it), algorithm confusion attacks (e.g., tricking a system expecting an RS256 signature into verifying an HS256 signature using the public key as the secret), using weak secrets for HMAC-based signatures (HS256), failing to validate expiration times, insecure transmission (not over HTTPS), or insecure client-side storage.[21]
  - **Relevance:** This applies if the chosen solution (typically a custom REST API implementation or potentially a plugin) uses JWTs to maintain the user's gated access state after the initial wallet verification. Lit Protocol's JWTs [36] use BLS signatures verified by its network, which mitigates some standard JWT risks like algorithm confusion, but the server *must* still correctly verify the signature and claims.[36]
  - **Mitigation:** Employ reputable, up-to-date JWT libraries. Always perform server-side validation of the

signature, the algorithm (alg header), and critical claims (exp, iss, aud).[21] Use strong, unique secrets for symmetric algorithms (HS256) or secure key management for asymmetric algorithms (RS256, ES256). Always set reasonable expiration times (exp claim). Transmit JWTs exclusively over HTTPS. Store JWTs securely on the client, often using HttpOnly cookies to prevent access via JavaScript.

- **Server-Side Request Forgery (SSRF)** [19]**:**
  - **Description:** SSRF occurs when an attacker can trick the server into making unintended requests to internal or external resources.[19] This can happen if an API fetches data from a URL provided or influenced by the user (e.g., fetching NFT metadata from a URI stored on-chain) without properly validating the target URL.
  - **Relevance:** Potentially relevant if the token gating logic involves the server fetching external resources based on data associated with the NFT or user input (e.g., resolving IPFS metadata URIs, calling external APIs based on token attributes) without strict validation and sanitization of the target URLs.
  - **Mitigation:** Implement strict input validation and sanitization for any user-supplied data used to construct URLs for server-side requests. Use allowlists to restrict requests to known, trusted domains. Avoid directly using raw URIs from untrusted sources like blockchain data without validation.
- OWASP API Security Top 10 Relevance 19:
  Several risks from the OWASP API Security Top 10 are directly applicable to token gating implementations, especially those using custom APIs:
  - **API1:2023 - Broken Object Level Authorization (BOLA):** Even if a user is authenticated (proven wallet ownership) and meets the basic gating requirement (e.g., owns *an* NFT from the collection), the system must ensure they are authorized to access the *specific* resource being requested. For instance, gating different pages based on different token IDs requires checking authorization for each specific page.
  - **API2:2023 - Broken Authentication:** Encompasses flaws in the core verification process – weak signature verification, improper nonce handling, insecure JWT validation, predictable session tokens, etc. Compromising authentication compromises the entire gating system.
  - **API3:2023 - Broken Object Property Level Authorization:** After successfully verifying access, the API should only return the necessary gated data, not excessive unrelated sensitive information.
  - **API5:2023 - Broken Function Level Authorization (BFLA):** An attacker who successfully bypasses gating for user-level content should not be able to escalate privileges to access administrative functions or modify gating rules. Proper role separation is key.
  - **API8:2023 - Security Misconfiguration:** Incorrect configuration of the web server, WordPress, plugins, JWT libraries (e.g., default secrets), or CORS settings [89] can create vulnerabilities.
  - **API9:2023 - Improper Inventory Management:** Failing to remove old, potentially vulnerable versions of gating plugins or custom API endpoints can leave attack surfaces exposed.

Security Evaluation of Solutions

Based on the potential vulnerabilities, the different approaches present varying security postures:

- **WordPress Plugins:**
  - *Client-Side Verification Focus:* Plugins or free tiers that appear to rely solely on client-side JavaScript checks (implied for Web3 Token Gate Free [23]) carry a **high risk** of bypass and are generally unsuitable for securing valuable content.[56]
  - *Server-Side Verification Focus:* Plugins claiming server-side checks (e.g., miniOrange [17], Unlock [25], Fungate [5]) offer **potentially moderate to high security**, but effectiveness depends entirely on the quality and correctness of their implementation. MiniOrange's mention of nonces [17] is positive. Fungate's reliance

on a proxy [5] introduces a component that needs scrutiny. Unlock's security is tied to its protocol infrastructure.[25] The closed-source nature of many premium plugins hinders independent security verification, requiring trust in the vendor.

- **Lit Protocol SDK:**
  - *Core Security:* The underlying use of threshold cryptography and decentralized verification across nodes provides a **strong theoretical security foundation**.[9] Trust is distributed rather than centralized.
  - *JWT Security:* The JWT mechanism [36], using BLS signatures verified against the network's public key, offers good protection against common JWT attacks *if* the server correctly performs the verification steps (checking signature and claims).[36]
  - *Overall:* Lit offers a **potentially high level of security** due to its decentralized architecture. However, the security of the *integration* still depends on the developer correctly implementing the server-side verification of Lit's responses (decryption results or JWTs).
- **Custom REST API:**
  - *Security Level:* The security is **highly variable**, ranging from **very low to very high**, depending entirely on the developer's expertise and diligence.[59] A well-implemented custom solution following all best practices (robust server-side signature and ownership checks, strong nonce strategy, secure session/JWT handling, secure library usage) can be very secure. Conversely, errors in any of these areas can lead to critical vulnerabilities.
  - *Key Risk Areas:* The implementation of ecrecover [79], nonce management, secure interaction with RPC nodes/APIs, and the security of chosen PHP libraries [97] are critical points of potential failure.

Best Practices for Server-Side NFT Ownership Verification [60]

Regardless of the overarching approach (plugin, SDK, custom), implementing secure server-side verification involves common best practices:

1. **Secure Authentication:** Always verify user control over a wallet address using a cryptographically signed message. This message *must* include a server-generated, single-use nonce to prevent replay attacks.[59]
2. **Server-Side Ownership Check:** Perform the actual check for NFT/token ownership on the backend server, *after* successful signature verification.[59]
3. **Use Trusted Data Sources:** Query blockchain state using a reliable method:
   - **Direct RPC Calls:** Connect to a trusted Ethereum node provider (e.g., Alchemy, QuickNode, Infura, or a self-hosted node) via their JSON-RPC endpoint.[61]
   - **Indexing APIs/NFT APIs:** Utilize specialized APIs (e.g., Alchemy NFT API [61], Moralis [64], QuickNode NFT API [63], OpenSea API [63], NFTPort [63], etc.) which often provide more efficient and aggregated ownership data compared to raw RPC calls.[60]
4. **Correct Contract Interaction:** Use the standard ERC functions: balanceOf(address) for ERC-721 [82], ownerOf(tokenId) for ERC-721 [82], and balanceOf(address, tokenId) or balanceOfBatch for ERC-1155.[82] Ensure interaction is with the verified contract address of the target NFT collection. EIP-165's supportsInterface can help confirm token standard.[109]
5. **Handle Caching Appropriately:** Server-side caching of ownership status can improve performance but must account for potential state changes (NFT transfers). If near real-time accuracy is critical, implement cache invalidation strategies, possibly using blockchain event monitoring or periodic re-verification.[111]
6. **Input Validation:** Sanitize and validate all inputs received from the client (address, signature, message,

nonce) and any data retrieved from the blockchain (e.g., metadata URIs) before using them in server-side logic.

Security Implications Summary

The security analysis yields critical takeaways for selecting and implementing a token gating solution:

- **Server-Side Verification is Essential for Security:** The inherent insecurity of client-side code execution [56] makes server-side verification a non-negotiable requirement for any token gating scenario involving valuable or sensitive content. The verification must happen in a controlled server environment, protected from user manipulation. OWASP principles reinforce this necessity.[19]
- **Authentication Must Precede Authorization:** Securely verifying *who* the user is (confirming control of the wallet address via signature and nonce [17]) must happen *before* checking *what* they own (verifying token balance/ownership [60]). Simply trusting a client-provided address is insecure.
- **Implementation Quality is Paramount:** Even when employing server-side checks, the actual security level is dictated by the quality of the implementation. Flaws in signature verification, nonce handling, library usage [21], API interaction, or session management can undermine the entire system.[22] This is particularly relevant for custom solutions where the developer holds full responsibility, but also applies to trusting the implementation quality of plugins.

Security Vulnerability Matrix for Token Gating Methods

The following table provides a comparative overview of the susceptibility of different approaches to key vulnerabilities:

Vulnerability	Client-Side Only Check	WP Plugin (Server-Side/API)	WP Plugin (Unlock Infra)	Lit Protocol SDK (JWT/Encrypt)	Custom REST API (Server-Side)
Client-Side Bypass [56]	Yes (High Risk)	No	No	No	No
Address Spoofing [101]	Partial (If no sig)	No (If sig used)	No (If sig used)	No (Relies on sig)	No (If sig used)
Signature Replay Attack [68]	N/A (No sig usually)	Partial (Depends on nonce)	Partial (Depends on nonce)	Partial (Depends on nonce/sig type)	Partial (Depends on nonce)
JWT Vulnerabilities [21]	N/A	Partial (If used by plugin)	N/A	Low (If server verifies Lit JWT correctly)	Partial (Depends on implementation)
SSRF [19]	No	Partial (Depends on impl.)	Partial (Depends on impl.)	Partial (Depends on impl.)	Partial (Depends on impl.)
BOLA/BFLA	N/A	Partial (Depends on impl.)	Partial (Depends on impl.)	Partial (Depends on impl.)	Partial (Depends on impl.)

(OWASP API) [19]					
Security Misconfiguration [19]	N/A	Yes (Plugin/Server config)	Yes (Plugin/Server config)	Yes (SDK/Server config)	Yes (Code/Server config)
Mitigation Difficulty	Very High (Fundamental)	Medium (Trust Vendor/Audit)	Medium (Trust Protocol)	Medium (Implement verification)	High (Full responsibility)
Overall Security Potential	Very Low	Moderate-High (Impl. Dep.)	Moderate-High (Impl. Dep.)	High (If integrated well)	Variable (Low to High)

Note: "Partial" indicates susceptibility depends heavily on the specific implementation details (e.g., correct use of nonces, JWT validation practices).

VII. Comparative Evaluation

Introduction

Having analyzed the functional and security aspects of various token gating approaches for WordPress, this section provides a structured comparison across key technical and operational criteria. The goal is to synthesize the findings into a comparative framework that highlights the strengths, weaknesses, and trade-offs of each solution category, thereby aiding technical decision-makers in selecting the most appropriate strategy for their specific context.

Evaluation Criteria Definition

The following criteria form the basis of this comparison:

- **Security Robustness:** Assesses the inherent security design and resistance to common vulnerabilities identified in Section VI. Key factors include reliance on server-side verification, strength of cryptographic methods (signature verification, nonce handling), protection against bypass and replay attacks, and overall architectural soundness (centralized vs. decentralized verification).
- **Ease of Implementation:** Considers the required technical skill level (PHP, JS, Web3, specific SDKs), estimated development time, complexity of the setup and configuration process, and the availability and clarity of documentation and examples.
- **Flexibility/Customizability:** Evaluates the degree to which the solution allows for defining sophisticated gating rules (e.g., conditions based on multiple tokens/collections, specific token attributes, quantities held, combined logic using AND/OR operators), control over the end-user experience (UI/UX), and the ability to integrate with other WordPress plugins or external systems.
- **Maintenance Overhead:** Encompasses the ongoing effort required to keep the solution functional and secure. This includes managing updates (for plugins, SDKs, or custom code), handling dependencies, monitoring for issues, and adapting to changes in underlying protocols or blockchain standards.
- **Cost (Development & Operational):** Includes upfront costs (developer time, premium plugin licenses) and ongoing operational expenditures (plugin subscription fees, potential network usage fees like Lit Protocol's Capacity Credits [42], API call charges from third-party indexers, server/hosting costs).
- **Community Support/Documentation:** Measures the availability and quality of supporting resources, such as official documentation, active community forums (Discord, GitHub Issues), responsiveness of developers or

support teams, and the existence of tutorials or code examples.

Comparative Analysis

Applying these criteria to the evaluated solutions yields the following comparative insights:

- **Free WP Plugins (e.g., Fungate, Web3 Token Gate Free):**
  - *Security:* Generally lower, especially if relying on client-side checks [23] or specific external APIs with potential proxy risks.[5] Verification transparency can be low.
  - *Ease of Implementation:* Often high for initial setup [5], but configuration might require external keys.[5]
  - *Flexibility:* Typically low, often limited to specific chains, token types, or gating simple content blocks.[23]
  - *Maintenance:* Variable; depends on developer activity. Open-source allows self-maintenance but requires expertise.[5]
  - *Cost:* Low initial cost (free software), but limitations may force upgrades or custom work later.
  - *Support/Docs:* Often limited to community forums or basic READMEs; support levels vary greatly.[5]
- **Premium WP Plugins (e.g., miniOrange, Unlock, Web3 Token Gate Pro):**
  - *Security:* Potentially moderate to high, often featuring server-side verification.[17] Security relies on vendor implementation quality. Closed-source nature limits independent audits.
  - *Ease of Implementation:* Generally easier than SDK or custom development, with integrated admin interfaces.[2]
  - *Flexibility:* Moderate to high, offering broader support for chains, wallets, token standards, and content types.[17] Integration with other plugins is common.[17]
  - *Maintenance:* Handled by the vendor through plugin updates. Requires keeping subscriptions active.
  - *Cost:* Involves subscription fees or one-time purchase costs.[17] Predictable operational cost.
  - *Support/Docs:* Typically includes dedicated support channels and more comprehensive documentation from the vendor.[28]
- **Lit Protocol SDK (Custom Integration):**
  - *Security:* Potentially high due to decentralized verification and strong cryptography.[9] Requires correct server-side validation of Lit responses.[36]
  - *Ease of Implementation:* High complexity, requiring significant JavaScript/Web3 development effort for SDK integration.[9] WP plugin is unreliable.[12]
  - *Flexibility:* Very high, enabled by flexible ACCs and Lit Actions for complex on-chain and off-chain conditions.[10]
  - *Maintenance:* Requires maintaining custom integration code and managing Lit SDK updates. Dependent on Lit network stability.
  - *Cost:* High development cost. Ongoing operational costs for network usage via Capacity Credits.[42] Pricing specifics need clarification.[11]
  - *Support/Docs:* Good official documentation and active developer community (Discord, GitHub).[34]
- **Zora (as Infrastructure for Custom Build):**
  - *Security:* Highly variable, entirely dependent on the custom server-side verification logic built by the developer.[59]
  - *Ease of Implementation:* High complexity, similar to custom REST API development. Requires building the full verification stack.[48]

- *Flexibility:* High, limited primarily by developer skill and standard EVM capabilities on Zora L2.[13]
- *Maintenance:* Requires maintaining the full custom codebase and managing interactions with Zora network/RPCs.
- *Cost:* High development cost. Potentially low operational cost due to L2 nature if on-chain interaction is needed [14], plus RPC/API costs.
- *Support/Docs:* Standard EVM/blockchain development resources apply. Limited Zora-specific guidance for this use case.[48]

- **Custom REST API (WordPress Backend):**
  - *Security:* Highly variable (Low to High). Depends entirely on developer implementation quality and security practices.[59] Full control, full responsibility.
  - *Ease of Implementation:* Very high complexity. Requires deep expertise in PHP/WordPress, Web3 signature verification, and security.
  - *Flexibility:* Maximum flexibility. Complete control over logic, UI/UX, and integrations.
  - *Maintenance:* Highest overhead. Requires maintaining the entire custom codebase, including any PHP libraries used.[72]
  - *Cost:* Very high development cost. Operational costs mainly involve server resources and potential third-party API/RPC usage fees.
  - *Support/Docs:* Relies on general WordPress, PHP, and Web3 documentation, plus specific library documentation. No dedicated support structure.

Key Trade-offs in Solution Selection

The comparison highlights fundamental trade-offs that decision-makers must weigh:

- **The Security vs. Ease of Use Spectrum:** A clear inverse relationship exists. Pre-built plugins offer the quickest path to implementation [1] but may compromise on security robustness or transparency, especially free versions potentially using client-side checks.[23] Conversely, the Lit Protocol SDK [11] and custom REST API development provide the potential for higher security and greater control, but demand substantially more development time, specialized expertise, and rigorous implementation.
- **Understanding the True Cost:** Evaluating cost requires looking beyond the initial software price tag. "Free" plugins [5] can incur significant hidden costs if their limitations necessitate premium upgrades [17] or extensive custom development workarounds. Custom solutions have high upfront development investments. Services like Lit Protocol introduce ongoing operational fees based on network usage.[42] A Total Cost of Ownership (TCO) perspective, encompassing development, potential subscriptions, operational fees, and maintenance effort, is crucial for accurate comparison.
- **Ecosystem Dependency vs. Control:** Opting for solutions like the Unlock Protocol plugin [1] or Fungate (with its Loopring dependency [5]) ties the implementation to those specific ecosystems and their infrastructure. Using Lit Protocol creates a dependency on its network and payment model.[42] While these ecosystems might offer benefits like shared infrastructure or network effects, they reduce architectural control. A custom REST API solution provides maximum independence and control but requires the developer to manage all components, including RPC access and the entire verification stack.

Comparative Evaluation Matrix of Token Gating Solutions

This matrix summarizes the relative strengths and weaknesses of each approach category:

Criterion	Free WP Plugin (Generic)	Premium WP Plugin (Generic)	Lit Protocol SDK (Custom)	Zora Infra + Custom Build	Custom REST API (WP Backend)
Security Robustness	Low-Medium (Risk of client-side checks, impl. varies)	Moderate-High (Server-side likely, vendor trust)	High (Decentralized verification, crypto strength)	Variable (Depends on custom code quality)	Variable (Depends on custom code quality)
Ease of Implementation	High	Medium-High	Low	Low	Very Low
Flexibility/Customizability	Low	Medium-High	Very High	High	Maximum
Maintenance Overhead	Low-Medium (Updates, potential self-fix)	Low (Vendor updates)	Medium (SDK updates, custom code)	High (Full custom code)	Very High (Full custom code)
Cost Model	Low Dev (Free SW), Hidden OpEx?	Med Dev (Config), Subscription OpEx	High Dev, Usage-based OpEx (Capacity Credits)	High Dev, RPC/API OpEx	Very High Dev, RPC/API OpEx
Support/Docs	Low-Medium (Community)	Medium-High (Vendor)	High (Protocol Team/Community)	Medium (General EVM/Zora)	Low (Self/General)

Note: Ratings are relative comparisons across the categories.

VIII. Strategic Mapping: Viable Solutions and Fallback Plans

Introduction

The analysis demonstrates that there is no single "best" solution for implementing token gating on WordPress. The optimal choice is highly dependent on the specific context of the project, balancing critical factors such as the required level of security, the budget allocated for development and operation, the technical expertise available within the team, and the desired degree of flexibility and customization. This section maps the evaluated solutions to common scenarios, providing recommendations for primary strategies and potential fallback options.

Scenario-Based Recommendations

- **Scenario 1: High Security & High Flexibility Required**
  - *(Use Case Example: Gating access to high-value premium content, financial data dashboards, or implementing complex, multi-conditional access rules for a sensitive community.)*
  - **Primary Strategy: Lit Protocol SDK (Custom Integration).** This approach is recommended due to its strong, decentralized security model based on threshold cryptography [9] and its inherent flexibility through Access Control Conditions (ACCs) and Lit Actions.[10] The JWT authentication flow [36] provides a robust mechanism for server-side enforcement without exposing the server to direct blockchain complexity. While the official Lit WP plugin appears unreliable [12], a custom integration allows leveraging the full power of the SDK. This requires significant JavaScript/Web3 development expertise and budget for potential

operational costs (Capacity Credits [42]).
  - **Fallback Strategy: Custom WordPress REST API Implementation.** If the complexity or operational cost model of Lit Protocol is prohibitive, a meticulously built custom solution using the WP REST API offers the next best alternative for control and security potential. This requires rigorous adherence to security best practices: server-side signature verification with nonces [17], secure PHP libraries [72], reliable server-side token ownership checks via trusted APIs/RPCs [60], and secure session or JWT management.[21] This path demands deep in-house expertise and thorough security auditing.
  - **Justification:** Both strategies prioritize security and control over ease of implementation. Lit offers unique decentralized verification benefits, while the custom API provides maximum architectural control, albeit with a higher security burden on the developer.
- **Scenario 2: Moderate Security & Ease of Implementation Prioritized**
  - *(Use Case Example: Basic members-only content for a community, gating access based on standard NFT collection ownership, moderate budget and development resources.)*
  - **Primary Strategy: Premium WordPress Plugin.** Select a reputable premium plugin such as miniOrange [17], the Unlock Protocol plugin [1], or Web3 Token Gate Pro.[23] The choice should be based on specific feature requirements (e.g., supported blockchains/wallets, WooCommerce integration needs [17]). Crucially, verify that the chosen plugin explicitly performs server-side verification of wallet ownership and token holdings.[17] These plugins offer a balance by providing necessary features and vendor support while reducing custom development effort.[28]
  - **Fallback Strategy: Carefully Vetted Free Plugin with Server-Side Checks.** If budget constraints are severe, consider a free plugin *only if* it demonstrably uses secure server-side verification. The Unlock Protocol plugin [25] fits this description if its feature set is sufficient. Fungate [5] could be considered *only* if the application is Loopring-specific and the security implications of its proxy mechanism [5] are fully understood and accepted. *Plugins relying solely on client-side verification should be strictly avoided.*
  - **Justification:** This approach balances the need for adequate security (via server-side checks) with the practical benefits of faster implementation and reduced development complexity offered by plugins. Premium options provide broader features and support. The fallback requires careful security assessment of free alternatives.
- **Scenario 3: Low Stakes & Quick Setup Needed**
  - *(Use Case Example: Simple proof-of-concept, gating non-sensitive promotional content, minimal budget and time.)*
  - **Primary Strategy: Free WordPress Plugin.** Options like the Unlock Protocol plugin [1], Fungate [5] (for Loopring), or Web3 Token Gate Free [23] can be used for rapid deployment. However, their limitations must be accepted: restricted features (chains, wallets, content types [23]), potential maintenance issues [12], and crucially, potentially weaker security models (especially if relying on client-side checks [23]).
  - **Fallback Strategy: Standard WordPress Controls.** If the requirement for Web3 token gating is not absolute, consider using built-in WordPress password protection for posts/pages or standard user role management plugins to control access. This avoids Web3 complexity and associated security risks entirely.
  - **Justification:** This scenario prioritizes speed and minimal cost above all else. The security risks associated with simpler free plugins or purely client-side checks might be acceptable only if the gated content is non-sensitive and the impact of unauthorized access is negligible.

Considerations for Selection

Beyond the scenarios, several factors should influence the final decision:

- **Team Expertise:** An honest assessment of the development team's skills in PHP, JavaScript, WordPress internals, Web3 protocols, SDK integration (like Lit's), and security implementation is crucial. Overestimating capabilities can lead to insecure or non-functional implementations, especially with custom solutions.
- **Budget:** Consider the full financial picture: upfront development costs, recurring plugin subscription fees, potential transaction or API usage fees (Lit Capacity Credits [42], third-party indexers), and ongoing maintenance costs.
- **Scalability Needs:** Evaluate how the chosen solution will handle growth in terms of the number of users, gated content items, complexity of gating rules, and required verification throughput.
- **Future Flexibility:** Anticipate potential future needs. Will support for new blockchains or token standards be required? Will more complex gating logic be necessary? Choosing a more flexible architecture (like Lit or a custom API) might offer better long-term adaptability, despite higher initial investment.

Strategic Implications

The mapping exercise highlights several strategic points:

- **Context is King: No Universal Solution:** The analysis confirms that the "best" token gating solution is not absolute but relative to the specific project's constraints and goals. A high-security financial application has vastly different requirements than a simple community blog, dictating different optimal approaches. The trade-offs identified in Section VII (Security vs. Ease, Cost, Control) must be carefully balanced.
- **Security Posture as a Foundational Decision:** The choice between relying on client-side verification versus implementing robust server-side verification is a fundamental architectural decision with profound security implications. As demonstrated in Section VI, client-side checks are easily circumvented [56], making server-side validation the necessary standard for any application where access control integrity matters.[59] This decision significantly influences which solutions are viable for a given risk tolerance.
- **The Importance of Fallback Planning:** The Web3 space is characterized by rapid evolution and potential instability. Plugins may become unmaintained [12], services might change pricing models [41] or encounter downtime, and custom implementations can reveal unforeseen bugs. Therefore, identifying a viable fallback strategy during the initial planning phase is a prudent measure. This might involve selecting a secondary plugin option, architecting custom code for easier migration, or having contingency plans for service dependencies.

IX. Conclusion

Summary of Findings

This report has conducted a technical analysis of various approaches for implementing NFT and token gating on WordPress websites. The evaluation covered dedicated WordPress plugins (both free and premium), the Lit Protocol SDK, the potential use of Zora's infrastructure, and custom development leveraging the WordPress REST API.

Key findings indicate a diverse landscape:

- **WordPress Plugins:** Offer a range of options from free, limited solutions often relying on client-side checks or specific external APIs [5], to feature-rich premium plugins typically providing server-side verification and broader compatibility but requiring subscriptions.[17] Open-source availability varies, as does maintenance status.[5]

- **Lit Protocol SDK:** Provides a powerful, decentralized approach using Access Control Conditions and threshold cryptography for flexible and potentially highly secure gating.[10] Its JWT mechanism enables robust server-side verification.[36] However, it demands significant development effort for custom integration (as the WP plugin seems unreliable [12]) and involves operational costs.[42]
- **Zora Infrastructure:** Functions as an L2 platform where gating *can be built* using standard EVM tools [13], but Zora does not appear to offer a dedicated, managed gating service/API.[48] Implementation requires developers to build their own server-side verification logic.
- **Custom REST API:** Affords maximum control and flexibility but requires the highest level of development expertise and places the entire security burden on the custom implementation.[59] The PHP Web3 library ecosystem requires careful navigation.[72]

Across all approaches, the critical importance of **server-side verification** for any meaningful security was consistently highlighted, contrasting sharply with the inherent vulnerabilities of client-side-only checks.[56] Secure authentication via signed messages with nonces is a prerequisite for reliable server-side checks.[17]

Key Recommendations

The optimal strategy for implementing token gating on WordPress depends heavily on project-specific requirements regarding security, flexibility, budget, and technical expertise. Based on the analysis, the following recommendations are provided:

1. **For Maximum Security and Flexibility:**
   - **Primary:** Pursue a custom integration using the **Lit Protocol SDK**. Leverage its decentralized verification, flexible ACCs/Lit Actions, and secure JWT flow for server-side enforcement. Budget for development time and operational costs (Capacity Credits).
   - **Fallback:** Develop a **Custom WordPress REST API solution**. This requires meticulous implementation, rigorous security testing, careful selection of PHP crypto libraries, and strong in-house expertise.
2. **For Balanced Security and Ease of Use:**
   - **Primary:** Select a reputable **Premium WordPress Plugin** (e.g., miniOrange, Unlock Protocol, Web3 Token Gate Pro) after confirming its use of server-side verification methods and ensuring its features meet project needs (chains, wallets, integrations).
   - **Fallback:** Consider a **Free Plugin** like Unlock Protocol *only if* its features suffice and its server-side verification model is deemed adequate. Thoroughly vet any free option's security claims.
3. **For Low-Stakes, Rapid Deployment:**
   - **Primary:** Utilize a **Free WordPress Plugin**, fully acknowledging its limitations in features, compatibility, and potential security weaknesses (especially if client-side checks are employed).
   - **Fallback:** Employ standard **WordPress password protection or user roles** if strict Web3 gating is not essential.

Final Thoughts

Implementing token gating introduces powerful capabilities but also significant technical and security challenges. Regardless of the chosen path – plugin, SDK, or custom code – ongoing security vigilance is essential. Thorough testing, regular updates (where applicable), and monitoring for emerging vulnerabilities are crucial. The dynamic nature of the Web3 landscape necessitates an adaptable strategy and awareness of potential changes in protocols, services, and best practices. Finally, educating end-users on secure wallet interaction practices [101] remains a vital component of the overall security posture for any Web3-enabled application. Careful consideration

of the specific use case, risk tolerance, and available resources will guide the selection of the most effective and secure token gating solution for any given WordPress project.

Works cited

1. Plugins & Integrations Archives - Unlock Guides, accessed on April 17, 2025, https://unlock-protocol.com/guides/category/plugins/
2. NFT Token Gating - miniOrange Documentation, accessed on April 17, 2025, https://developers.miniorange.com/docs/wordpress-web3/features/token-gating
3. NFT based gated content - miniOrange, accessed on April 17, 2025, https://www.miniorange.com/web3/nft-based-gated-content/
4. NFT token-gated membership management - miniOrange, accessed on April 17, 2025, https://www.miniorange.com/blog/nft-token-gated-membership-management/
5. stepwn/Fungate: Enable Web3 Login and Token Gating ... - GitHub, accessed on April 17, 2025, https://github.com/stepwn/Fungate
6. Nft-token-gating Plugins — WordPress.com, accessed on April 17, 2025, https://wordpress.com/plugins/browse/nft-token-gating/
7. Token Plugins - WordPress.com, accessed on April 17, 2025, https://wordpress.com/plugins/browse/token/
8. Nft Plugins - WordPress.com, accessed on April 17, 2025, https://wordpress.com/plugins/browse/nft/
9. Token gate videos with Lit - Livepeer Docs, accessed on April 17, 2025, https://docs.livepeer.org/developers/tutorials/token-gate-videos-with-lit
10. Encryption and Access Control - Lit Protocol, accessed on April 17, 2025, https://developer.litprotocol.com/concepts/access-control-concept
11. Encryption and Access Control | Lit Protocol, accessed on April 17, 2025, https://developer.litprotocol.com/sdk/access-control/intro
12. Token / NFT / Blockchain Page Gating Plugin - WordPress.com, accessed on April 17, 2025, https://wordpress.com/plugins/litprotocol-wp-lit-gated
13. Deploying Contracts - ZORA Docs, accessed on April 17, 2025, https://docs.zora.co/zora-network/contracts
14. Zora is now LIVE on Formo!, accessed on April 17, 2025, https://formo.so/blog/zora-is-now-live-on-formo
15. Zora: Redefining Value in Digital Art - Gate.io, accessed on April 17, 2025, https://www.gate.io/learn/articles/zora-redefining-value-in-digital-art/4879
16. WordPress REST API: How to Access, Enable, & Use It (With Examples) - Jetpack, accessed on April 17, 2025, https://jetpack.com/resources/wordpress-rest-api/
17. Web3 – Crypto wallet Login & NFT token gating - WordPress.org, accessed on April 17, 2025, https://wordpress.org/plugins/web3-authentication/
18. WalletConnect - GitHub, accessed on April 17, 2025, https://github.com/walletconnect
19. OWASP Top 10 API Security Risks – 2023, accessed on April 17, 2025, https://owasp.org/API-Security/editions/2023/en/0x11-t10/
20. OWASP Top 10 API Security Vulnerabilities | Curity, accessed on April 17, 2025,

https://curity.io/resources/learn/owasp-top-ten/
21. Testing JSON Web Tokens - OWASP Foundation, accessed on April 17, 2025, https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens
22. Transaction Authorization - OWASP Cheat Sheet Series, accessed on April 17, 2025, https://cheatsheetseries.owasp.org/cheatsheets/Transaction_Authorization_Cheat_Sheet.html
23. Web3 Token Gate Plugin - WordPress.com, accessed on April 17, 2025, https://wordpress.com/plugins/web3-token-gate
24. Web3 – Crypto wallet Login & NFT token gating Plugin - WordPress.com, accessed on April 17, 2025, https://wordpress.com/plugins/web3-authentication
25. unlock-protocol/unlock-wordpress-plugin: A plugin for ... - GitHub, accessed on April 17, 2025, https://github.com/unlock-protocol/unlock-wordpress-plugin
26. Guide to the Unlock Protocol WordPress Plugin - Unlock Guides, accessed on April 17, 2025, https://docs.unlock-protocol.com/move-to-guides/plugins-and-integrations/wordpress-plugin
27. accessed on January 1, 1970, https://github.com/LIT-Protocol/lit-wp-lit-gated
28. NFT Token Gating for Ethereum on WordPress - Plugins - miniOrange, accessed on April 17, 2025, https://plugins.miniorange.com/ethereum-wordpress-nft-integration
29. Web3 – Crypto wallet Login & NFT token gating - WordPress, accessed on April 17, 2025, https://lv.wordpress.org/plugins/web3-authentication/
30. WordPress Web3 NFT Token Gate & Crypto Wallet Connect - Plugins - miniOrange, accessed on April 17, 2025, https://plugins.miniorange.com/web3-wordpress-login
31. Web3 – Crypto wallet Login & NFT token gating - WordPress.org Lithuania, accessed on April 17, 2025, https://lt.wordpress.org/plugins/web3-authentication/
32. WordPress Plugin for Auth0 Authentication - GitHub, accessed on April 17, 2025, https://github.com/auth0/wordpress
33. How Does Lit Protocol Work, accessed on April 17, 2025, https://developer.litprotocol.com/resources/how-it-works
34. Lit Protocol: Overview, accessed on April 17, 2025, https://developer.litprotocol.com/
35. Lit Protocol Use Cases - Notion, accessed on April 17, 2025, https://litprotocol.notion.site/Lit-Protocol-Use-Cases-a94916becdc0411f848c3095722c7864
36. JWT Auth | Lit Protocol, accessed on April 17, 2025, https://developer.litprotocol.com/sdk/access-control/jwt-auth
37. 1. Intro - Lit Protocol, accessed on April 17, 2025, https://developer.litprotocol.com/learninglab/intro-to-lit/intro
38. lit-js-sdk 1.3.4 | Documentation - Lit Protocol, accessed on April 17, 2025, https://serrano-sdk-docs.litprotocol.com/

39. LIT Live Price Chart, Market Cap & News Today - CoinGecko, accessed on April 17, 2025, https://www.coingecko.com/en/coins/lit
40. Lit Protocol Price | LITKEY Price Today, Live Chart, USD converter, Market Capitalization | CryptoRank.io, accessed on April 17, 2025, https://cryptorank.io/price/lit-protocol
41. Lit Protocol Announces Pre-Sale for Native Network Token $LITKEY, accessed on April 17, 2025, https://www.prnewswire.com/news-releases/lit-protocol-announces-pre-sale-for-native-network-token-litkey-302401034.html
42. Paying for Usage of Lit | Lit Protocol, accessed on April 17, 2025, https://developer.litprotocol.com/paying-for-lit/overview
43. Capacity Credits | Lit Protocol, accessed on April 17, 2025, https://developer.litprotocol.com/paying-for-lit/capacity-credits
44. Litentry price today, LIT to USD live price, marketcap and chart | CoinMarketCap, accessed on April 17, 2025, https://coinmarketcap.com/currencies/litentry/
45. Tokenomics - Help Centre - Zora, accessed on April 17, 2025, https://support.zora.co/en/articles/4797185
46. Zora Airdrop Guide: All You Need to Know About the $Zora Airdrop - QuickNode Blog, accessed on April 17, 2025, https://blog.quicknode.com/zora-airdrop-guide-all-you-need-to-know-about-the-zora-airdrop/
47. Permissions | ZORA Docs, accessed on April 17, 2025, https://docs.zora.co/contracts/Permissions1155
48. ZORA Docs, accessed on April 17, 2025, https://docs.zora.co/
49. Building Token Metadata - ZORA Docs, accessed on April 17, 2025, https://docs.zora.co/protocol-sdk/metadata/token-metadata
50. How to Mint on Zora with an App - Base Docs, accessed on April 17, 2025, https://docs.base.org/cookbook/use-case-guides/creator/nft-minting-with-zora
51. Best NFT API for Zora Data - SimpleHash, accessed on April 17, 2025, https://simplehash.com/chains/zora
52. ZORA API Hackathon API Submissions - Gist - GitHub, accessed on April 17, 2025, https://gist.github.com/erikreppel/7a884ae83d5a6d3fffdf37a8ce3a79b1
53. Token Gating on Solana - A Solana Mobile Tutorial - Helius, accessed on April 17, 2025, https://www.helius.dev/blog/token-gating-on-solana-mobile-tutorial
54. Understanding zkOS: Client-Side vs. Server-Side ZK - Aleph Zero, accessed on April 17, 2025, https://alephzero.org/blog/client-side-vs-server-side-zero-knowledge/
55. PKCE authentication: client-side vs server-side | Kristóf Marussy, accessed on April 17, 2025, https://marussy.com/pkce-authentication/
56. Client-Side vs. Server-Side Security | Blog - Digital.ai, accessed on April 17, 2025, https://digital.ai/catalyst-blog/client-side-vs-server-side-security/
57. OWASP Top 10 Client-Side Security Risks, accessed on April 17, 2025, https://owasp.org/www-project-top-10-client-side-security-risks/
58. Authentication and Authorization - zotregistry.dev, accessed on April 17, 2025, https://zotregistry.dev/v2.1.0/articles/authn-authz/

59. How to make a website with NFT authentication? : r/ethdev - Reddit, accessed on April 17, 2025, https://www.reddit.com/r/ethdev/comments/x060n8/how_to_make_a_website_with_nft_authentication/
60. NFT Ownership API - Bitquery V2 API Docs | Blockchain Data API (V2), accessed on April 17, 2025, https://docs.bitquery.io/docs/examples/nft/nft-ownership-api/
61. How to Check the Owner of an NFT - Alchemy Docs, accessed on April 17, 2025, https://docs.alchemy.com/docs/how-to-check-the-owner-of-an-nft
62. AMB Access Polygon - Developer Guide - AWS Documentation, accessed on April 17, 2025, https://docs.aws.amazon.com/managed-blockchain/latest/ambp-dg/amb-polygon-dg.pdf
63. Top 10 Ethereum NFT APIs - 101 Blockchains, accessed on April 17, 2025, https://101blockchains.com/top-ethereum-nft-apis/
64. Web3 APIs - How to Build dApps Faster with Web3 and NFT APIs, accessed on April 17, 2025, https://second-pocket-shoot-73.hashnode.dev/web3-apis-how-to-build-dapps-faster-with-web3-and-nft-apis
65. Server-Side vs. Client-Side Tracking: Which Is Better? - Session Interactive, accessed on April 17, 2025, https://sessioninteractive.com/blog/server-side-tracking-vs-client-side-tracking/
66. Authenticating to the REST API - GitHub Docs, accessed on April 17, 2025, https://docs.github.com/en/rest/authentication/authenticating-to-the-rest-api
67. reown-com/web-examples: Wallet and dapp examples implementing WalletConnect v2 - GitHub, accessed on April 17, 2025, https://github.com/reown-com/web-examples
68. Automatic Authentication Signature - Wallets - Fellowship of Ethereum Magicians, accessed on April 17, 2025, https://ethereum-magicians.org/t/automatic-authentication-signature/2429
69. WordPress - Custom REST API endpoint example - GitHub Gist, accessed on April 17, 2025, https://gist.github.com/24380108940011017e6c
70. WordPress REST API Custom Endpoint Example - GitHub, accessed on April 17, 2025, https://gist.github.com/anthonycoffey/271dc89dfc7bcb8bd424edf07e5c1d5e
71. docs/extending-the-rest-api/adding-custom-endpoints.md at master · WP-API/docs - GitHub, accessed on April 17, 2025, https://github.com/WP-API/docs/blob/master/extending-the-rest-api/adding-custom-endpoints.md
72. simplito/elliptic-php: Fast, general Elliptic Curve Cryptography library. Supports curves used in Bitcoin, Ethereum and other cryptocurrencies (secp256k1, ed25519, ..) - GitHub, accessed on April 17, 2025, https://github.com/simplito/elliptic-php
73. Simplito - GitHub, accessed on April 17, 2025, https://github.com/simplito
74. Repositories - Simplito - GitHub, accessed on April 17, 2025, https://github.com/orgs/simplito/repositories

75. README.md - simplito/bn-php - GitHub, accessed on April 17, 2025, https://github.com/simplito/bn-php/blob/master/README.md
76. PHP library to validate an ethereum signature - GitHub, accessed on April 17, 2025, https://github.com/agustind/php-ethereum-signature-validation
77. php ecrecover helper and examples - GitHub, accessed on April 17, 2025, https://github.com/wmh/php-ecrecover
78. Verifying the sign messages secp256k1 in PHP - Reddit, accessed on April 17, 2025, https://www.reddit.com/r/PHP/comments/sbsb7s/verifying_the_sign_messages_secp256k1_in_php/
79. Why doesn't my verification of a signed message signature work in PHP?, accessed on April 17, 2025, https://ethereum.stackexchange.com/questions/39945/why-doesnt-my-verification-of-a-signed-message-signature-work-in-php
80. digitaldonkey/ecverify: PHP based Ethereum web3 personal recover - GitHub, accessed on April 17, 2025, https://github.com/digitaldonkey/ecverify
81. digitaldonkey/ethereum-php: PHP interface to Ethereum JSON-RPC API. Fully typed Web3 for PHP 7.X - GitHub, accessed on April 17, 2025, https://github.com/digitaldonkey/ethereum-php
82. Ultimate Guide to NFT Software Development in 2024 - Rock'n'Block, accessed on April 17, 2025, https://rocknblock.io/blog/nft-software-development-ultimate-guide
83. How to find all the NFTs, ERC721 and ERC1155, owned by address of *a certain collection*? - Ethereum Stack Exchange, accessed on April 17, 2025, https://ethereum.stackexchange.com/questions/150998/how-to-find-all-the-nfts-erc721-and-erc1155-owned-by-address-of-a-certain-col
84. How to check if user owns ERC1155 token from specific contract?, accessed on April 17, 2025, https://ethereum.stackexchange.com/questions/121800/how-to-check-if-user-owns-erc1155-token-from-specific-contract
85. ERC1155 check if token owner - Contracts - OpenZeppelin Forum, accessed on April 17, 2025, https://forum.openzeppelin.com/t/erc1155-check-if-token-owner/8503
86. WordPress REST API Authentication Using Fetch - Stack Overflow, accessed on April 17, 2025, https://stackoverflow.com/questions/46204166/wordpress-rest-api-authentication-using-fetch
87. Guide to Understanding JWT Validation (+Code Examples) - Criipto, accessed on April 17, 2025, https://www.criipto.com/blog/jwt-validation-guide
88. WordPress REST API JWT Authentication Method - Plugins - miniOrange, accessed on April 17, 2025, https://plugins.miniorange.com/wordpress-rest-api-jwt-authentication-method
89. JWT Authentication for WP REST API – WordPress plugin, accessed on April 17, 2025, https://wordpress.org/plugins/jwt-authentication-for-wp-rest-api/
90. WordPress REST API Authentication Using Third Party Provider - Plugins -

miniOrange, accessed on April 17, 2025, https://plugins.miniorange.com/wordpress-rest-api-authentication-using-third-party-provider
91. Issue #97 · Tmeister/wp-api-jwt-auth - Custom Endpoint - GitHub, accessed on April 17, 2025, https://github.com/Tmeister/wp-api-jwt-auth/issues/97
92. WebDevStudios/WDS-WP-REST-API-Connect: A tool for connecting to the JSON-based REST API for WordPress via OAuth - https://github.com/WP-API/OAuth1, https://github.com/WP-API/OAuth1 - GitHub, accessed on April 17, 2025, https://github.com/WebDevStudios/WDS-WP-REST-API-Connect
93. I made a custom rest endpoint in my wordpress site where i can make a post request from an app i made. How do i make this secure? - Reddit, accessed on April 17, 2025, https://www.reddit.com/r/Wordpress/comments/wtarxy/i_made_a_custom_rest_endpoint_in_my_wordpress/
94. Verifying Ethereum (Web3) signed message in PHP - Stack Overflow, accessed on April 17, 2025, https://stackoverflow.com/questions/52957249/verifying-ethereum-web3-signed-message-in-php
95. zbkm/siwe: Ethereum Authentication for PHP - GitHub, accessed on April 17, 2025, https://github.com/zbkm/siwe
96. Install PHP GMP for elliptic curve cryptography - Laracasts, accessed on April 17, 2025, https://laracasts.com/discuss/channels/php/install-php-gmp-for-elliptic-curve-cryptography
97. Simplito Elliptic-php 1.0.2 security vulnerabilities, CVEs, accessed on April 17, 2025, https://www.cvedetails.com/version/1448479/Simplito-Elliptic-php-1.0.2.html
98. PHP Fatal error: Uncaught Error: Class 'Elliptic\EC' not found - Stack Overflow, accessed on April 17, 2025, https://stackoverflow.com/questions/61551198/php-fatal-error-uncaught-error-class-elliptic-ec-not-found
99. walletconnect-docs/docs/api/sign/wallet-usage.md at main - GitHub, accessed on April 17, 2025, https://github.com/WalletConnect/walletconnect-docs/blob/main/docs/api/sign/wallet-usage.md
100. About tokengating - Shopify.dev, accessed on April 17, 2025, https://shopify.dev/docs/apps/build/blockchain/tokengating
101. Crypto: Protect Yourself From Address Poisoning - Brandsec, accessed on April 17, 2025, https://www.brandsec.com.au/crypto-protect-yourself-from-address-poisoning/
102. Address Poisoning Attacks and Trezor, accessed on April 17, 2025, https://trezor.io/support/a/address-poisoning-attacks
103. Common crypto scams and how to avoid them - Phantom, accessed on April 17, 2025, https://phantom.com/learn/crypto-101/common-crypto-scams

104. Dusting attacks: watch out for your crypto wallets - Blog, accessed on April 17, 2025, https://blog.1inch.io/dusting-attacks-watch-out-for-your-crypto-wallets/
105. authentication - JWT vs. Client Certificates - Information Security Stack Exchange, accessed on April 17, 2025, https://security.stackexchange.com/questions/128185/jwt-vs-client-certificates
106. Token-gated Application Architecture - Unlock Protocol, accessed on April 17, 2025, https://docs.unlock-protocol.com/tutorials/building-token-gated-applications/
107. How to secure JSON-RPC endpoint with a NFT token? - Ethereum Stack Exchange, accessed on April 17, 2025, https://ethereum.stackexchange.com/questions/127694/how-to-secure-json-rpc-endpoint-with-a-nft-token
108. How to Get All NFTs Owned by an Address - Alchemy Docs, accessed on April 17, 2025, https://docs.alchemy.com/docs/how-to-get-all-nfts-owned-by-an-address
109. How to check if the token on opensea is ERC721 or ERC1155 using node.js, accessed on April 17, 2025, https://stackoverflow.com/questions/69706835/how-to-check-if-the-token-on-opensea-is-erc721-or-erc1155-using-node-js
110. How to Create an NFT Allowlist Based on Ownership - Alchemy Docs, accessed on April 17, 2025, https://docs.alchemy.com/docs/how-to-create-allowlists-based-on-nft-ownership
111. How to Create a Secure NFT Gated Website | QuickNode Guides, accessed on April 17, 2025, https://www.quicknode.com/guides/ethereum-development/dapps/how-to-create-a-secure-nft-gated-website